REACT

SESSION 1 • COMPLETE NOTES

React + Vite • JSX • Components • Props • Rendering

Learning Goal
Create a React app with Vite and understand React, JSX, components, imports, JavaScript in JSX, functions, lists, objects, booleans, props, children and conditional rendering.

Prerequisite: Basic HTML, CSS and JavaScript. Course uses JavaScript/JSX, not TypeScript.

01 Installing React with Vite

Definition: Vite is the development/build tool used to scaffold, run and build our React project.

Step 1: Check Node.js and npm

TERMINAL

```
node -v
npm -v
```

Step 2: Create a Vite project

TERMINAL

```
npm create vite@latest
```

Choose React as framework and JavaScript as variant.

Step 3: Install and run

TERMINAL

```
cd react-session-1
npm install
npm run dev
```

Vite prints a local development URL, commonly http://localhost:5173/.

Do: use npm create vite@latest. Don't: use old Create React App setup instructions for this course.

02 What is Vite?

Definition: Vite is a frontend build tool providing a fast development server and a production build process.

Hinglish: React UI banata hai; Vite project ko develop/run/build karne ka environment deta hai.

Tool	Main job
React	Build user interfaces from components.
Vite	Development server and build tooling.
npm	Package manager and script runner.
Node.js	Runtime used by development tooling.

COMMON COMMANDS

```
npm run dev
npm run build
npm run preview
```

Remember: React and Vite are not the same thing.

03 What is React?

Definition: React is a JavaScript library for building user interfaces from reusable components.

Hinglish: UI ko small reusable components mein todkar application build karte hain.

REACT COMPONENT

```
function ProductCard() {
  return (
    <div>
      <h2>JavaScript Book</h2>
      <p>Price: ₹499</p>
      <button>Buy Now</button>
    </div>
  );
}
```

Why React? Reusability, component-based structure, declarative UI, and easier organization of interactive applications.

> Important
> React JavaScript ka replacement nahi hai. JavaScript React ki foundation hai.

04 First React Code

Open src/App.jsx. A minimal React component:

App.jsx

```
function App() {
  return <h1>Hello React!</h1>;
}

export default App;
```

Breakdown: function = component, return = UI, JSX = UI syntax, export = make it available to other files.

FIRST PRACTICE

```
function App() {
  const name = "Student";

  return (
    <div>
      <h1>Welcome to React</h1>
      <p>Hello {name}</p>
    </div>
  );
}

export default App;
```

Don't: custom component names should start with an uppercase letter, e.g. App.

05

What is a Component?

Definition: A component is a reusable piece of UI represented by a JavaScript function that returns JSX/React elements.

COMPONENT TREE

```
App
├── Navbar
├── Hero
├── ProductList
│   ├── ProductCard
│   ├── ProductCard
│   └── ProductCard
├── Cart
└── Footer
```

Benefits: reusability, maintainability, readability and composition.

06 Creating and Importing Components

Create: src/components/Header.jsx

Header.jsx

```
function Header() {
  return (
    <header>
      <h1>My Website</h1>
    </header>
  );
}

export default Header;
```

Import and use:

App.jsx

```
import Header from "./components/Header";

function App() {
  return (
    <div>
      <Header />
      <p>Welcome.</p>
    </div>
  );
}

export default App;
```

Do: PascalCase names such as Header.jsx and ProductCard.jsx. Don't: forget export/import.

07

Why className Instead of class and htmlFor Instead of for?

Definition: JSX uses JavaScript/DOM-style property names for these attributes.

HTML	JSX
class="card"	className="card"
for="email"	htmlFor="email"

CORRECT JSX

```
function Login() {
  return (
    <div className="card">
      <label htmlFor="email">Email</label>
      <input id="email" type="email" />
    </div>
  );
}
```

WRONG / AVOID

```
// Avoid in JSX
<div class="card">...</div>
<label for="email">Email</label>
```

08 Implementing JavaScript in React / JSX

Definition: Put JavaScript expressions inside JSX using { }.

JSX + JAVASCRIPT

```
function App() {
  const name = "Punit";
  const age = 20;

  return (
    <div>
      <h1>Hello {name}</h1>
      <p>Age: {age}</p>
      <p>Next year: {age + 1}</p>
    </div>
  );
}
```

Commonly used inside { }: variables, expressions, function calls, ternaries and object properties.

WRONG

```
// Wrong
return (
  <div>
    const name = "Punit";
    <h1>{name}</h1>
  </div>
);
```

09

Using Functions in React

Definition: React components are JavaScript functions. Helper functions and event handler functions can also be used.

HELPER FUNCTION

```
function getFullName(firstName, lastName) {
  return firstName + " " + lastName;
}

function App() {
  const name = getFullName("Rahul", "Kumar");
  return <h1>{name}</h1>;
}
```

EVENT HANDLER

```
function App() {
  function handleClick() {
    alert("Button clicked!");
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

DO / DON'T

```
// Wrong: calls immediately
<button onClick={handleClick()}>Click</button>

// Correct: pass the function
<button onClick={handleClick}>Click</button>
```

10 Using map() to Make a List

Definition: map() transforms every array item. In React it is commonly used to generate repeated JSX.

ARRAY → JSX

```
function App() {
  const students = ["Aman", "Riya", "Karan"];

  return (
    <ul>
      {students.map((student) => (
        <li key={student}>{student}</li>
      ))}
    </ul>
  );
}
```

OBJECT ARRAY

```
const products = [
  { id: 1, name: "Keyboard", price: 999 },
  { id: 2, name: "Mouse", price: 499 },
];

function App() {
  return (
    <div>
      {products.map((product) => (
        <div key={product.id}>
          <h2>{product.name}</h2>
          <p>₹{product.price}</p>
        </div>
      ))}
    </div>
  );
}
```

Key: Prefer a stable unique ID from your data instead of array index when a real ID exists.

11 Objects in React

Definition: React works with normal JavaScript objects. Render specific properties with dot notation.

OBJECT + JSX

```
function App() {
  const user = {
    name: "Punit",
    age: 20,
    city: "Kaithal",
    role: "Student",
  };

  return (
    <div>
      <h1>{user.name}</h1>
      <p>Age: {user.age}</p>
      <p>City: {user.city}</p>
      <p>Role: {user.role}</p>
    </div>
  );
}
```

NESTED OBJECT

```
const student = {
  name: "Riya",
  contact: {
    email: "riya@example.com",
    city: "Kaithal",
  },
};

function App() {
  return <p>{student.contact.email}</p>;
}
```

Don't: {user} ko directly render mat karo. Specific property use karo, or debugging ke liye {JSON.stringify(user)}.

12 Why Can't We Render Boolean Directly?

Key fact: React treats true, false, null and undefined as empty children. They do not appear as visible text.

BOOLEAN OUTPUT

```
function App() {
  const isLoggedIn = true;
  const isAdmin = false;

  return (
    <div>
      <p>{isLoggedIn}</p>
      <p>{isAdmin}</p>
    </div>
  );
}
```

If you want the words true/false:

DISPLAY AS TEXT

```
const isLoggedIn = true;
return <p>{String(isLoggedIn)}</p>;
```

More commonly, booleans are used as conditions:

CONDITION

```
const isLoggedIn = true;

return (
  <div>
    {isLoggedIn && <p>Welcome back!</p>}
  </div>
);
```

Pitfall
0 is renderable. So {count && <Message />} can show 0 when count is 0. Use {count > 0 && ...} when needed.

13 Props

Definition: Props are data passed from a parent component to a child. They can contain strings, numbers, booleans, arrays, objects, functions and JSX.

PASSING PROPS

```
function App() {
  return (
    <Student
      name="Aman"
      age={20}
      course="React"
    />
  );
}
```

RECEIVING PROPS

```
function Student({ name, age, course }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>Course: {course}</p>
    </div>
  );
}

export default Student;
```

OBJECT PROP

```
const student = { name: "Riya", age: 21 };

<Student student={student} />

function Student({ student }) {
  return <h2>{student.name}</h2>;
}
```

Remember: Props are read-only from the receiving component's perspective.

14 children Props

Definition: JSX placed between a component's opening and closing tags is received through the children prop.

CHILDREN PROP

```
function Card({ children }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}

function App() {
  return (
    <Card>
      <h2>React Course</h2>
      <p>Learn React step by step.</p>
    </Card>
  );
}
```

Useful for reusable wrappers such as Cards, Panels, Modals and Layouts.

15 Conditional Rendering

Definition: Conditional rendering means showing different JSX according to a condition. React uses normal JavaScript conditions.

IF / ELSE

```
function App() {
  const isLoggedIn = true;
  let content;

  if (isLoggedIn) {
    content = <h2>Welcome back!</h2>;
  } else {
    content = <h2>Please log in.</h2>;
  }

  return <div>{content}</div>;
}
```

TERNARY

```
function App() {
  const isLoggedIn = true;

  return (
    <div>
      {isLoggedIn ? <h2>Welcome</h2> : <h2>Login</h2>}
    </div>
  );
}
```

&&

```
function App() {
  const isAdmin = true;

  return (
    <div>
      <h1>Dashboard</h1>
      {isAdmin && <button>Admin Panel</button>}
    </div>
  );
}
```

Do: choose the simplest readable condition. Don't: use deeply nested ternaries that turn the UI into a puzzle.

16 Final Assignment

Complete every task in the same React + Vite project. Write the code yourself and test each requirement in the browser.

Task 1	Vite Setup: Create a React + Vite JavaScript project, install dependencies and run the dev server.
Task 2	First React Code: Build a personal introduction using at least three JavaScript variables and JSX.
Task 3	Components: Create Navbar, Profile and Footer components inside src/components and import them into App.jsx.
Task 4	className + htmlFor: Build a styled profile card and an email label/input using className, htmlFor and id.
Task 5	Functions: Create getGreeting() and an onClick handler that displays an alert.
Task 6	map(): Create five student objects and render them with map(); use each student's unique id as key.
Task 7	Objects: Create a user object with name, age, city and skills. Render the properties and map the skills.
Task 8	Boolean Rendering: Render a boolean directly and observe it. Then show Available/Not Available conditionally and show Admin only for admins.
Task 9	Props: Build ProductCard and pass name, price, category and availability. Render at least three products.
Task 10	children: Build Card using children and use it for About, Skills and Contact sections.
Task 11	Conditional Rendering: Build Login Status with logged-in/logged-out UI and an Admin Panel condition.
Task 12	Mini Challenge: Build a Student Directory combining components, props, objects, map(), functions, children and conditional rendering. Each student has name, course, city, skills and isActive.

Student Checklist

I can…	Done
Create React with Vite	☐
Explain React vs Vite	☐
Create/import components	☐
Use className/htmlFor	☐
Use JS inside JSX	☐
Use functions/events	☐
Render arrays with map()	☐
Use objects	☐
Explain boolean rendering	☐
Pass props	☐
Use children	☐
Use conditional rendering	☐
Complete Student Directory	☐

Official references used

Current setup and syntax were checked against the official React and Vite documentation: react.dev and vite.dev. The notes follow the documented patterns for components, JSX, props, children, lists, conditional rendering, and Vite scaffolding.

Next session
JSX in depth: JSX rules, expressions, fragments, attributes, JavaScript inside JSX, and common JSX mistakes.